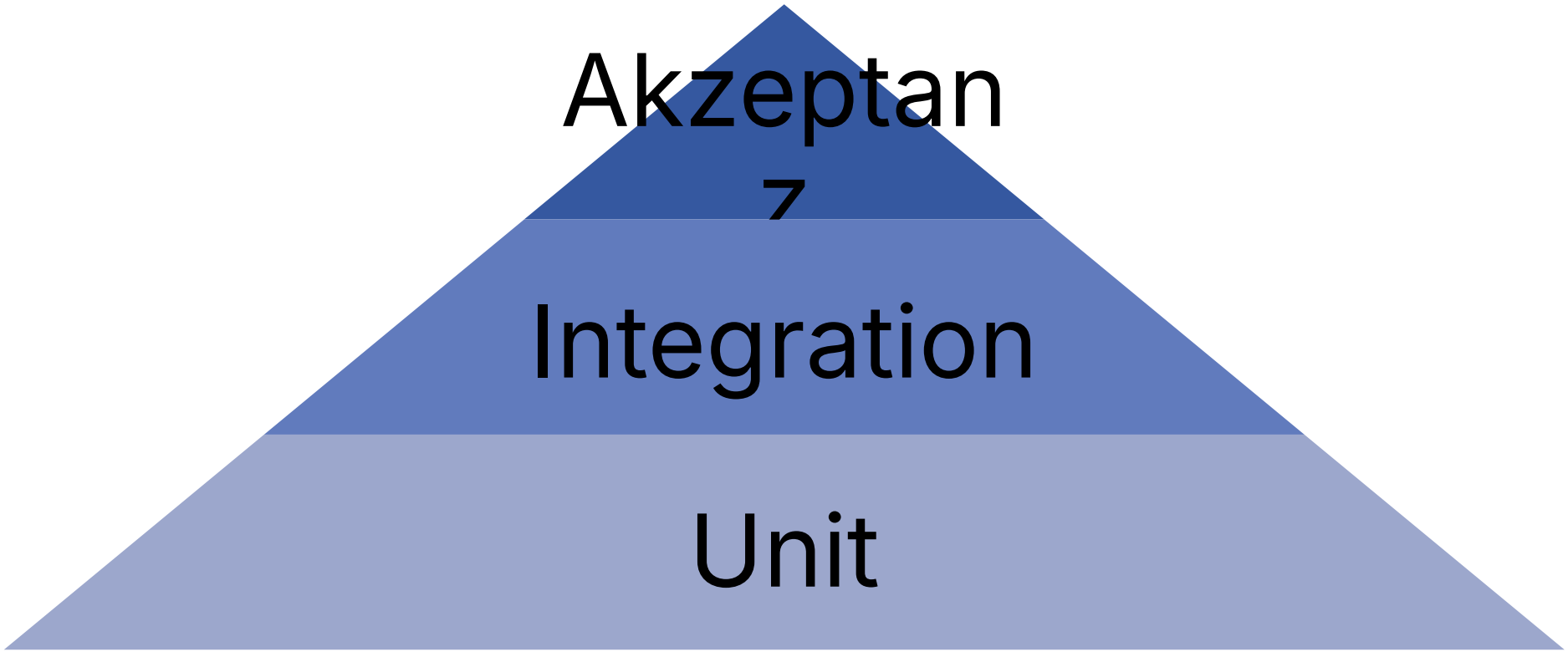
Testing

Unit Tests (Vitest)

End-to-End (Cypress / Playwright)



Testpyramide

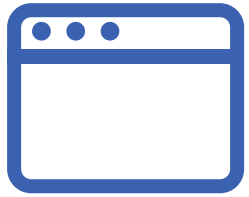


Angular Test-Runner: Vitest (Default ab v21)



Vitest

Test-Runner und Framework: startet die Tests, stellt Mocks und Matcher bereit, meldet die Ergebnisse



jsdom

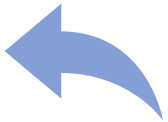
Simulierte Browser-Umgebung: Komponenten laufen im Test ohne echten Browser

Vitest – Standard-Test-Runner ab Angular 21

```
// Neue Projekte (ng new) und alle Übungsprojekte nutzen Vitest
// angular.json
"test": {
  "builder": "@angular/build:unit-test",
  "options": { "tsConfig": "tsconfig.spec.json", "runner": "vitest" }
}

// Start: ng test   (einmalig: ng test --watch=false)
// describe / it / expect / beforeEach / afterEach: wie gewohnt
// Mocks und Spies: vi.fn() und vi.spyOn()
// Bestandsprojekte mit Karma/Jasmine laufen weiter (Umstieg: siehe nächste Folien)
```

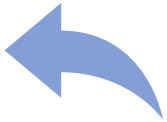
Konfiguration Vitest package.json



```
"devDependencies": {
  ..
  "@angular/build": "^21.0.0",
  "jsdom": "^28.0.0",
  "typescript": "~5.9.0",
  "vitest": "~4.0.18",
  ..
}
```

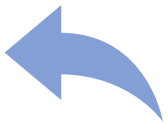
Konfiguration Vitest angular.json

```
"test": {
  "builder": "@angular/build:unit-test",
  "options": {
    "tsConfig": "tsconfig.spec.json",
    "runner": "vitest"
  }
}
```



Konfiguration Vitest tsconfig.spec.json

```
/* To learn more about this file see:
https://angular.dev/reference/configs/file-structure. */
{
  "extends": "./tsconfig.json",
  "compilerOptions": {
    "outDir": "./out-tsc/spec",
    "types": [
      "vitest/globals"
    ]
  },
  "include": [
    "src/**/*.spec.ts",
    "src/**/*.d.ts"
  ]
}
```



Umstieg von Jasmine auf Vitest (Bestandsprojekte)

// Jasmine / Karma → Vitest

jasmine.createSpy() → vi.fn()

jasmine.createSpyObj('S', ['a']) → { a: vi.fn() }

spyOn(obj, 'm') → vi.spyOn(obj, 'm')

spy.and.returnValue(x) → spy.mockReturnValue(x)

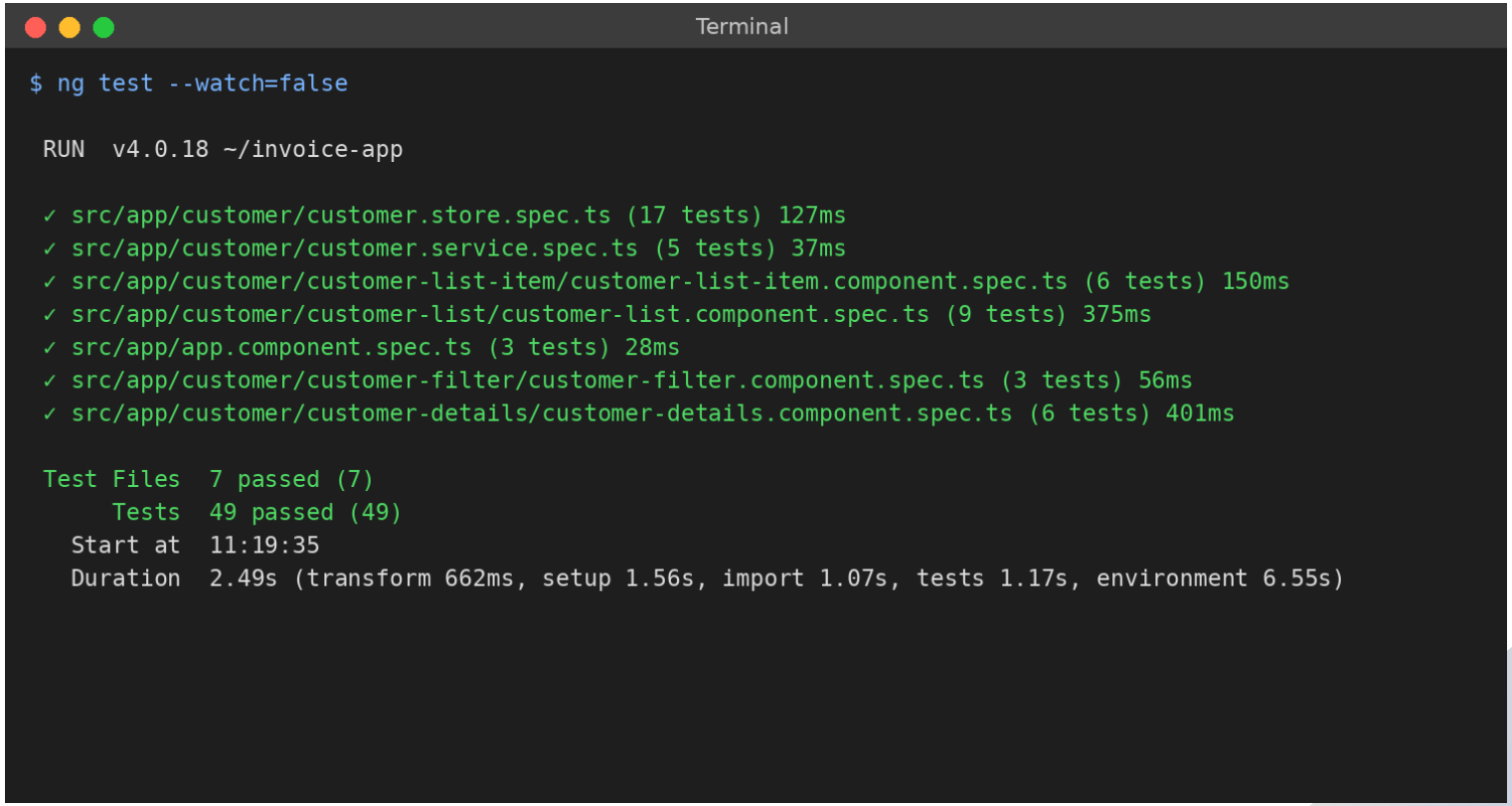
spy.and.callFake(fn) → spy.mockImplementation(fn)

spy.calls.count() → spy.mock.calls.length

expect(list).toHaveSize(2) → expect(list).toHaveLength(2)

it(..., (done) => { ...; done(); }) → it(..., async () => { await ...; })

// karma.conf.js, src/test.ts, polyfills.ts entfallen; Typen: "vitest/globals"



Test Runner

> ng test

Vitest Vokabular

Suites – describe(name, fn)

- Eine Suite bündelt zusammengehörige Tests (describe-Block)

Tests (Specs) – it(name, fn)

- 1 Test bestehend aus Titel
- und einer Funktion mit dem Test → Erwartungen(Expectations).

Expectations expect(actual).toBe(expected)

- Erwartung - technische Implementierung deines Tests → true oder false auswertet.

Matchers

- vordefinierte Hilfsfunktionen für Erwartungen, z. B.: toBe, toEqual, toContain, toHaveLength, toHaveBeenCalledWith.
- komplette Liste: vitest.dev/api/expect

Einfaches Beispiel – Testsubjekt ExampleService

```
describe('ExampleService', () => {
  let service: ExampleService;


  beforeEach(() => {
    TestBed.configureTestingModule({});
    service = TestBed.inject(ExampleService);
  });

  it('should be created', () => {
    expect(service).toBeTruthy();
  });
});
```

Suite

Setup

Expectation

Matcher

Setup, Teardown

beforeEach
afterEach

- einmal pro Test (it)

beforeAll
afterAll

- einmal pro Test-Suite (describe)

Setup initialisiere Testwelt

- beforeEach(() ⇒ {
  TestBed.configureTestingModule({}); Setup TestModule mittels Config Object
  service = ***TestBed***.inject(ExampleService);
  });

Expectation

-

  Beschreibung

- it('should be created', () ⇒ {
    expect(service).toBeTruthy();
  });

  Testcode mit matcher

UnitTest Component

```
@Component({
  selector: 'app-greeter',
  templateUrl: './greeter.component.html',
  styleUrl: './greeter.component.css'
})
export class GreeterComponent {
  name = signal('');
}
```

- <h1>Hello {{name()}}</h1>

10/11/2026

UnitTests Component - Setup

```
let component: GreeterComponent;
let fixture: ComponentFixture<GreeterComponent>;   // Teststeuerobjekt für Component
beforeEach(() => {
  TestBed.configureTestingModule({
    imports: [GreeterComponent]   // Standalone Component wird importiert
  });
  fixture = TestBed.createComponent(GreeterComponent);
  component = fixture.componentInstance;
  componentRootElement = fixture.nativeElement;   // HTML Element der Component
  debugElement = fixture.debugElement;
  fixture.detectChanges();   // ChangeDetection Trigger
});
```

UnitTests Component – Set

```
it('should show Hello <name>', async () => {
    component.name.set('Tester');   // Signal setzen
    fixture.detectChanges();   // Change Detection auslösen
    await fixture.whenStable();   // Warte bis alle Async Calls beendet sind
    expect(componentRootElement.querySelector('h1')?.textContent).toBe('Hello Tester');
    expect(debugElement.query(By.css('h1')).nativeElement.textContent).toBe('Hello
    Tester');
});
```

Dependency Injection

```
export class GreeterComponent implements
OnInit {
  name = signal('');
  private readonly service =
inject(GreeterService);
  public ngOnInit(): void {
    this.name.set(this.service.requestName());
  }
}
```

```
@Injectable({ providedIn: 'root' })
export class GreeterService {
  public requestName(): string {
    return 'some example name';
  }
}
```

10/11/2026

Mocking Dependencies

```
// Ersatzobjekt mit Mock-Funktion statt echtem Service
const greeterServiceSpy = { requestName: vi.fn() };
beforeEach(() => {
  TestBed.configureTestingModule({
    imports: [GreeterComponent],
    providers: [
      { provide: GreeterService, useValue: greeterServiceSpy }
    ]
  });
  // Rückgabewert festlegen
  greeterServiceSpy.requestName.mockReturnValue('Tester');
  fixture = TestBed.createComponent(GreeterComponent);
});
```

Async result

```
@for (name of names(); track name) {
  <p>{{name}}</p>
}
```

```
export class GreeterComponent {
  names = signal<string[]>([]);
  private readonly service =
inject(GreeterService);
  public requestNames(): void {
    this.service.requestNames().subscribe(result
⇒ this.names.set(result));
  }
}
```

```
export class GreeterService {

  public
requestNames():Observable<string[]>{
    return //HTTP CALL...
  }
}
```

Aync result

```
greeterServiceSpy.requestNames.mockReturnValue(of(['Tester1', 'Tester2']));
...
it('should show nothing at the beginning', async () => {
  fixture.detectChanges();
  await fixture.whenStable();
  expect(componentRootElement.querySelector('p')).toBeNull();
});

it('after requesting data 2 p-elements should be visible', async () => {
  component.requestNames();
  fixture.detectChanges();
  await fixture.whenStable();
  expect(componentRootElement.querySelectorAll('p')).toHaveLength(2);
});
```

Throw Error

```
greeterServiceSpy.requestNames.mockReturnValue(throwError(() ⇒ new
Error('TestError')));
```

Trigger Eventhandling

```
@for (name of names(); track name) {
  <p (click)="setSelectedName(name)">{{name}}</p>
}

selected = output<string>();

public setSelectedName(name: string): void {
  this.selected.emit(name);
}
```

```
let selectedName = '';
component.selected.subscribe(name => (selectedName = name));
const allPElements = fixture.debugElement.queryAll(By.css('p'));
allPElements.at(0)?.triggerEventHandler('click');
expect(selectedName).toEqual('Tester1');
```

Übung 1

- Teste die Klasse CustomerListComponent
  - Mocke den Service mit vi.fn() und gib 0, 1, 2 Kunden zurück
  - Prüfe ob die Liste <ul> korrekt aufgebaut
- Teste die currentActiveCustomers Pipe
  - Prüfe mit verschiedenen Werten für „daysInPast“

Cypress

End2End Tests

Installation Cypress

ng e2e

Cannot find "e2e" target for the specified project.

You can add a package that implements these capabilities.

For example:

Cypress: ng add @cypress/schematic

Nightwatch: ng add @nightwatch/schematics

WebdriverIO: ng add @wdio/schematics

Puppeteer: ng add @puppeteer/ng-schematics

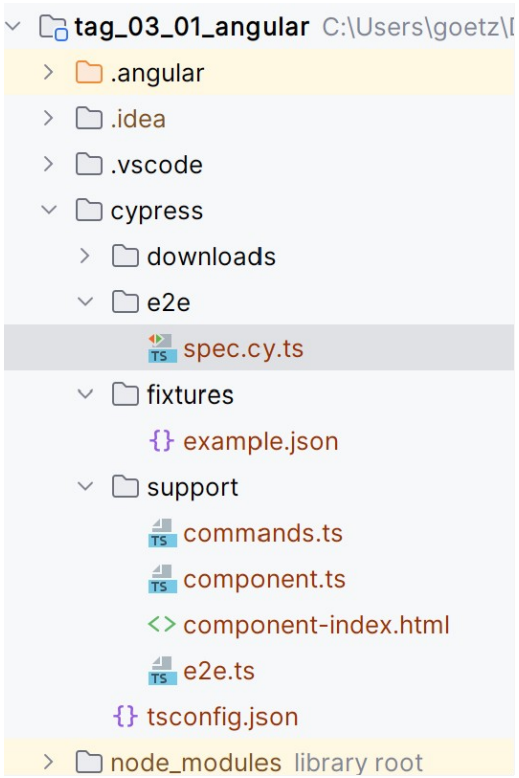
Playwright: ng add playwright-ng-schematics

Would you like to add a package with "e2e" capabilities now? Cypress

i Using package manager: npm

...

Konfig


tag_03_01_angular C:\Users\goetz\
.angular
.idea
.vscode
cypress
downloads
e2e
spec.cy.ts
fixtures
example.json
support
commands.ts
component.ts
component-index.html
e2e.ts
tsconfig.json
node_modules library root

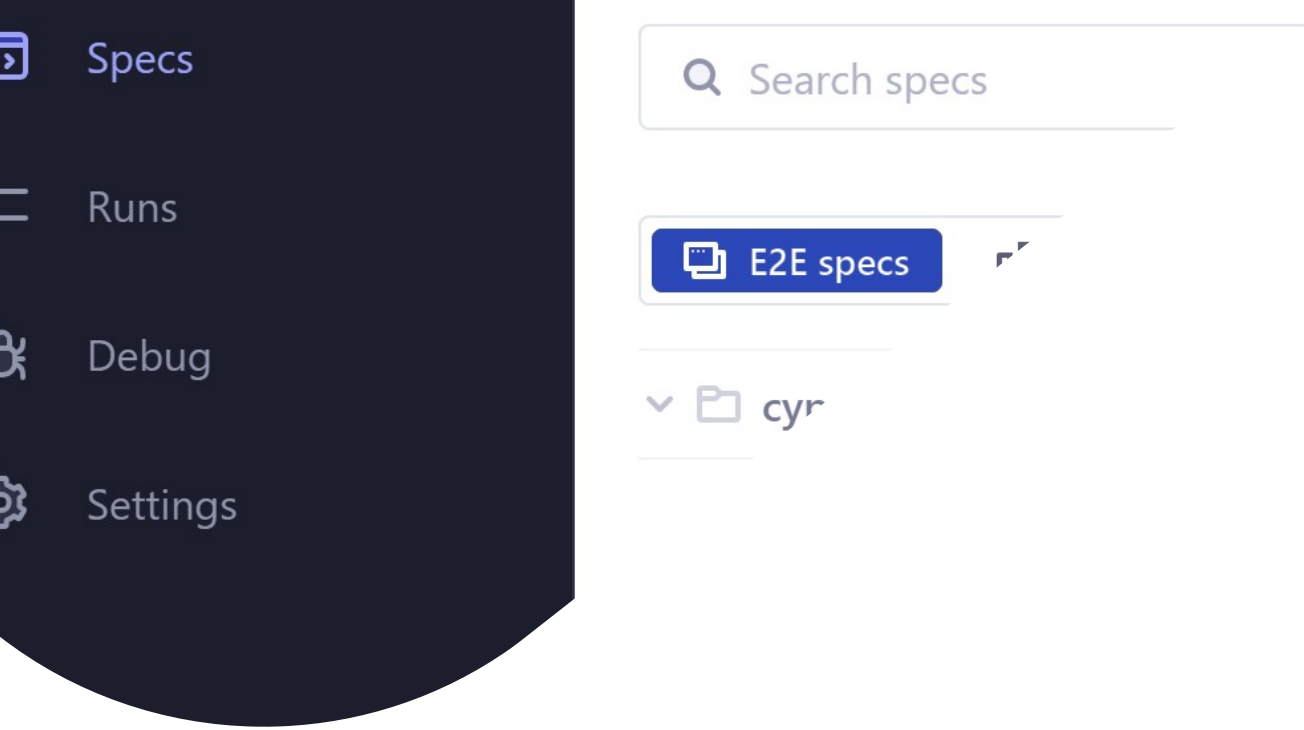
Bestandteile eines Tests

```
describe('My First Test', () => {                          Suite

  it('Visits the initial project page', () => {            Specs

    cy.visit('/‘)
    cy.pause();
   cy.get("li").first().should("have.class","selected");    Matcher, Expectation


  })
})
```



Übung 2

- Installieren sie Cypress mittels ng e2e
- Passen sie die generierte Datei spec.cy.ts
- Ändern sie
  - cy.contains('app ís running‘) in
  - cy.contains('Invoice Manager App') ab

- Starten sie ng e2e und navigiere zu
  - spec.cy.ts

Cypress Elements – DSL

- Elemente definieren die structure und Verhalten einer Seite.
- Durch das selektieren und interagieren mit diesen, kann man funktionale Tests schreiben.
- Beispiel ID Selektor
  - cy.get('#username').type('user@angularseminar.com‘);
  - cy.get('#password').type('password‘);
- Beispiel Attribut Selektor
  - cy.get('[formControlName=username]').type('user@angularseminar.com‘);
- Beispiel Element Selektor
  - cy.get('ul');

Cypress Elements – Element API

Funktion	Bedeutung
.type()	Ausfüllen eines Textfeldes
.click()	Klicken / Selektieren
.first()	Falls element NodeListe, gib erstes
.focus()	Setzt den Fokus auf das Element
.should()	Matcher siehe nächste Folie

Assertions und Matchers

- Die Funktion should() an einem Element leitet den Prüfschritt ein

- cy.get("appCustomerList").should("exist");
- cy.get(".selected").should("not.exist");
- cy.get(".selected").should("be.visible");
- cy.get("li").first().should("have.class","selected");
- cy.get("li").first().should("not.have.value",„Hans");
- cy.url().should("contain","greeting");

Übung 3

- Erstellen Sie eine neue Test Methode navigation_works
- Hier soll der erste Eintrag in der CustomerListe geklickt werden
- Danach soll geprüft werden, dass
  - Die App auf der Detailseite ist
    - Mittels URL Matcher
    - Und prüfen ob lastOrder Date angezeigt wird